PRIMER PARCIAL PROGRAMACIÓN DE COMPUTADORES III

PLAN DETRABAJO

INTEGRANTES:

WILDER PABLOVERDECIA DIAZ

ERICK JHOAN FUENTES CELEDON

LEONEL EDUARDOGARRIDO NAEDER

DOCENTE: ALFREDO BAUTISTA

CONTEXTUALIZACIÓN DEL PROBLEMA

La empresa de transporte intermunicipal TransCesar S.A.S. opera rutas entre municipios del departamento del Cesar, Colombia. Actualmente, toda su operación de venta de tiquetes se gestiona de forma completamente manual: los pasajeros deben presentarse físicamente en la terminal el mismo día en que desean viajar para adquirir su tiquete, sin ningún tipo de garantía de disponibilidad de cupos.

Este modelo operativo genera múltiples inconvenientes: largas filas en la terminal, insatisfacción de los clientes al no encontrar cupos disponibles en fechas de alta demanda (especialmente fines de semana y días festivos), pérdida de ingresos por pasajeros que optan por otras empresas, y ausencia de información consolidada sobre las ventas realizadas.

A medida que el proyecto avanza, se identifican nuevas necesidades: la gestión formal de rutas, el módulo de reservas anticipadas, reglas de negocio avanzadas (límite de tiquetes por día, tarifas festivas) y reportes con filtros.

OBJETIVO GENERAL

Diseñar e implementar un sistema de gestión de tickets intermunicipales para TransCesar S.A.S. utilizando los principios de Programación Orientada a Objetos y Arquitectura en Capas en Java, que permita administrar vehículos, conductores, pasajeros, rutas, tiquetes y reservas, con persistencia en archivos de texto plano

ROLES DE CADA INTEGRANTE:

Lider: Erick Jhoan Fuentes Celedon

Desarrollador 1: Wilder Pablo Verdecia Diaz

Desarrollador 2: Leonel Eduardo Garrido Naeder

RAMAS Y DIVISIÓN DEL TRABAJO

Para iniciar el desarrollo se crearon tres ramas: **feature/menu-principal**, desarrollada por el Lider

En esta rama se desarrolla el menú principal del sistema, es decir la interfaz con la que el usuario va a interactuar cuando ejecute el programa, esta rama pertenece a la capa view y se conecta con la capa de servicio Trabaja en la clase:

- Menu.java
- Main.java

El líder solo debe enfocarse en este apartado ya que el resto de desarrolladores se encargaran de la estructura completa del sistema

Al ser el líder se encarga de ver que todo este bajo control y que todo este saliendo de acuerdo a los planes, cualquier duda se comunica a el para solucionarla en conjunto

feature/clase-vehiculo, desarrollada por el Desarrollador 1

En esta rama se desarrolla el apartado de los vehículos (Modelos, Dao, Service), se crean las clases necesarias para que esta parte del sistema funcione, las clases respectivas son:

- Vehiculo.java
- Bus.java
- Buseta.java
- Microbus.java
- ServicioVehiculo.java
- VehiculoDAO.java

Como agregado también se trabajan las interfaces:

- Calculable.java
- Imprimible.java

El desarrollador 1 se asegura de que exista una conexión entre las capas, DAO y Service para que el líder pueda conectar la capa Service con el menú e implementar el apartado de Vehiculo en el sistema **feature/clase-persona**, desarrollada por el Desarrollador 2 En esta rama se desarrolla el apartado de las personas (Modelos, Dao, Service), se crean las clases necesarias para que esta parte del sistema funcione, las clases respectivas son:

- Persona.java
- Pasajero.java
- PasajeroAdultoMayor.java
- PasajeroEstudiante.java
- PasajeroRegular.java
- PasajeroService.java
- PasajeroDAO.java

Como agregado también al igual que el desarrollador 1 se trabajan las interfaces:

- Calculable.java
- Imprimible.java

El desarrollador 2 se asegura de que exista una conexión entre las capas, DAO y Service para que el líder pueda conectar la capa Service con el menú e implementar el apartado de Vehiculo en el sistema

Una vez establecido los roles de cada persona, cada uno se enfocó en su rama y empezó el desarrollo del sistema

REQUERIMIENTOS EXTRAS

El sistema de gestión de tickets de TransCesar S.A.S. ha sido bien recibido por la empresa. Sin embargo, tras las primeras semanas de uso, el equipo directivo ha identificado nuevas necesidades operativas que el sistema actual no cubre, por lo cual se deben incorporar estos nuevos requerimientos es posible sin reescribir lo que ya existe. Cada mejora afecta únicamente la capa que le corresponde, dejando las demás intactas.

La nueva misión como desarrolladores es extender el sistema actual implementando los requerimientos descritos a continuación, aplicando el mismo flujo de trabajo en Git.

Dicho esto, nuevamente se dividieron los roles, para que cada uno trabaje de la mejor manera posible

Lider: Se encarga de añadir al menú principal los nuevos requerimientos que van a trabajar los desarrolladores 1 y 2

Desarrolladores 1 y 2: Trabajan en conjunto para poder desarrollar el sistema con la mejor eficiencia posible, es posible que ambos corrijan los errores del uno y del otro, pero para esto es necesario la supervisión del líder Esta no es la única problemática que surge como requerimiento extra, también se presenta el
siguiente caso:

El sistema de gestión de tickets de TransCesar S.A.S. ha evolucionado significativamente gracias al trabajo del equipo de desarrollo. El registro de vehículos, conductores, pasajeros y la venta de tickets ya funciona correctamente. Sin embargo, la empresa ha identificado una necesidad crítica que actualmente genera pérdida de clientes: no existe forma de reservar un puesto con anticipación.

Hoy en día un pasajero que quiere viajar el fin de semana debe presentarse físicamente en la terminal el mismo día para comprar su ticket, sin ninguna garantía de que haya cupos disponibles. Esto genera filas, inconformidad y en muchos casos el pasajero simplemente busca otra empresa de transporte.

La solución que propone la empresa es implementar un módulo de reservas que permita a los pasajeros apartar un cupo con anticipación. El sistema debe garantizar que el cupo quede reservado temporalmente mientras el pasajero decide si confirma el viaje, con reglas claras de expiración y cancelación para evitar que los vehículos queden bloqueados con reservas que nunca se convierten en tickets.

Para solucionar esta ultima problemática, por ultima vez el equipo se pone manos a la obra a trabajar en ello

El líder vuelve a supervisar el trabajo y se encarga de actualizar el menú principal con los nuevos requerimientos mientras que por otro lado los desarrolladores vuelven a trabajar en conjunto para ir añadiendo lo que necesite el sistema

A continuación se presentan las actividades mas en especifico que hizo cada integrante del grupo

LIDER

Se creo el repositorio en github el día 4 de marzo, el día 5 se creo el primer método en la clase Main, como líder se dió supervisión a todo el trabajo realizado por los dos desarrolladores y los días siguientes se siguio trabajando en el menú de usuario según lo necesitado y los nuevos requerimientos, se realizaron los merge a la rama main mientras se seguía trabajando en el menú yla documentación necesaria

DESARROLLADOR 1

Inicio el desarrollo el 10 de marzo de 2026 Se fijo en la rama feature/clase-vehiculo y allí empezó el desarrollo haciendo un commit para crear
las capas y la clase madre Vehiculo de la cual saldrían el resto de subclases

Al día siguiente el 11 de marzo se añadieron los setters y getters junto con métodos para la clase vehiculo y sus subclases

El 15 de marzo de 2026 se añadieron los requerimientos para la rama entera, empezando con la creación del Service y del Dao de vehículos los cuales son necesarios para realizar las validaciones y almacenar la información del sistema, también se creo la interfaz Imprimible se implemento en las clases para poder ver la información, finalmente se trabajo en el Service y el Dao solucionando problemas y conectando las capas El 21 de marzo de 2026 vuelve el desarrollo del sistema para el desarrollador 1, iniciando con la
actualización de métodos para el nuevo apartado de rutas que se añadió, también se trabajo en colaboración con el líder para poder crear los apartados del menú y que estos funcionen adecuadamente, esto se fue haciendo a lo largo del día con tiempos rondando desde 15 minutos a máximo 2 horas

DESARROLLADOR 2

Inicio del desarrollo el 15 de marzo de 2026

Se trabajó en la rama feature/clase-vehiculo donde se comenzó el desarrollo del sistema. Se realizaron varios commits durante la tarde: primero se implementó la jerarquía de clases persona y tipos de pasajero, luego se agregó la interfaz Calculable, seguido de la implementación de la clase Ticket con el cálculo de tarifa. Posteriormente se implementó la capa DAO para persistencia en archivos y la capa Service para lógica de negocio, finalizando el día con un cambio en la estructura del switch.

El 16 de marzo de 2026 se realizó una corrección de import ya que la interfaz estaba en el mismo package.

El 21 de marzo de 2026 se retomó el desarrollo con varios commits durante la mañana: se integró la clase Ruta y se actualizó Vehiculo para usarla ajustando la lógica de negocio, luego se agregaron las clases Reserva, ReservaDAO, GestorReservas y el método buscarPorCedula. Se realizó también un fix cambiando el nombre de ReservadaoDAO a ReservaDAO. Continuando el día se actualizaron PasajeroDAO y PasajeroService agregando validación de edad mínima de 60 años para el registro de adulto mayor, persistencia del tipo de pasajero en archivo y corrección del método buscarPorCedula para reconstruir correctamente cada tipo de pasajero. Finalizando el día en la tarde con la adición de la clase de festivos junto con sus validaciones.

DIAGRAMA UML

Por ultimo pero no menos importante se creo el diagrama UML para que se pueda visualizar como se interconectan las clases y sus métodos entre si

La arquitectura sigue un patron en tres capas bien definidas: Presentacion (Menu/Main), Logica de Negocio (Servicios) y Persistencia (DAOs con archivos planos), lo que facilita el mantenimiento y la escalabilidad del sistema.

Aspecto	Detalle
Tipo de sistema	Gestion de transporte publico (buses,microbuses, busetas)
Arquitectura	3 capas: Presentacion, Servicios, Persistencia (archivos)
Lenguaje objetivo	Java (tipos LocalDate, List<>, double, boolean)
Patron de diseño	DAO + Service Layer + Herencia con polimorfismo
Total de clases	22 clases e interfaces identificadas en el diagrama
Interfaces	Imprimible, Calculable
Clases abstractas	Vehiculo (jerarquia de vehiculos)

El sistema esta organizado en capas con responsabilidades claramente separadas. A continuacion se describe cada capa y las clases que la componen.

Capa	Clases	Responsabilidad
Presentacion	Main, Menu	Punto de entrada. Interaccion con el usuario via consola.
Servicios	GestorReservas, TicketService, PasajeroService, ServicioVehiculo	Logica de negocio, validaciones y orquestacion de operaciones.
Dominio	Vehiculo, Bus, MicroBus, Buseta, Pasajero, PasajeroRegular, PasajeroAdultoMayor, PasajeroEstudiante, Persona, Ruta, Reserva, Ticket, Festivos	Entidades del negocio con sus atributos y comportamientos.
Persistencia	VehiculoDAO, ReservaDAO, TicketDAO, PasajeroDAO, FestivosDAO	Lectura y escritura de datos en archivos de texto.
Contratos	Imprimible, Calculable	Interfaces que definen comportamientos obligatorios.

El diagrama define dos interfaces que establecen contratos de comportamiento para las clases que las implementen.

INTERFAZ IMPRIMIBLE

Define el contrato de impresion. Cualquier clase que la implemente debe proporcionar una representacion imprimible desus datos.

Metodo	Retorno	Descripcion
imprimir()	void	Imprime la informacion del objeto en consola o archivo.

INTERFAZ CALCULABLE
Define el contrato de calculo de tarifas. Obliga a las clases implementadoras a proveer logica de calculo del total.

Metodo	Retorno	Descripcion
calcularTotal()	double	Calcula y retorna el valor total(tarifa, descuento, etc.).

La jerarquia de vehiculos sigue el patron de herencia con clase abstracta en la raiz. Vehiculo define la estructura y comportamiento comun, mientras que Bus, MicroBus y Buseta especializan el calculo de tarifas y la obtencion del tipo.

CLASE ABSTRACTA VEHICULO
Clase base de todos los vehiculos del sistema. Implementa Imprimible y Calculable.

Atributos (protegidos)

Atributo	Tipo	Descripcion
tarifa	float	Tarifa base del vehiculo.
capacidad	int	Numero maximo de pasajeros que puede transportar.
ruta	Ruta	Ruta asignada al vehiculo (objeto Ruta).
velocidad	int	Velocidad del vehiculo.
estado	String	Estado actual: activo, inactivo, en ruta, etc.
conductor	Persona	Persona asignada como conductor.
placa	String	Identificador unico del vehiculo.

Metodos publicos

Metodo	Retorno	Descripcion

getTipoVehiculo()	String	Retorna el tipo: Bus, MicroBus o Buseta.
getEstado()	String	Retorna el estado actual del vehiculo.
calcularTarifaRecargo(double)	double	Calcula tarifa aplicando recargo (p.ej. festivos).
abordarPasajero()	Persona	Registra el abordaje de un pasajero.
getCapacidad()	int	Retorna la capacidad maxima.
setCapacidad()	void	Modifica la capacidad maxima.
getRuta()	Ruta	Retorna la ruta asignada.
setRuta()	void	Asigna una nueva ruta al vehiculo.
isActivo()	boolean	Indica si el vehiculo esta operativo.
getPlaca()	String	Retorna la placa del vehiculo.
imprimir()	void	Imprime informacion del vehiculo.

CLASE BUS

Subclase concreta de Vehiculo para buses de mayor capacidad.

Metodo	Retorno	Descripcion
calcularTarifaRecargo(double)	double	Logica especifica de recargopara Bus.
imprimirDetalles()	void	Imprime detalles extendidosdel Bus.
fromArchivo(String)	Bus	Construye un Bus desde unalinea del archivo de datos.
getTipoVehiculo()	String	Retorna el literal "Bus".

CLASE MICROBUS

Subclase concreta de Vehiculo para microbuses de capacidad intermedia.

Metodo	Retorno	Descripcion
getTipoVehiculo()	String	Retorna el literal "MicroBus".
calcularTarifaRecargo(double)	double	Logica especifica de recargopara MicroBus.
imprimirDetalles()	void	Imprime detalles extendidosdel MicroBus.
fromArchivo(String)	MicroBus	Construye un MicroBus desdeel archivo de datos.

CLASE BUSETA
Subclase concreta de Vehiculo para busetas de menor capacidad.

Metodo	Retorno	Descripcion
calcular()	double	Implementacion del calculo detarifa para Buseta.
imprimir()	void	Implementacion de impresionpara Buseta.

El sistema modela la jerarquia de personas con herencia. Persona es la clase base, Pasajero la extiende con datos de viaje, y tres tipos especializados de pasajero implementan distintas logicas de descuento.

CLASE PERSONA
Clase base para cualquier persona en el sistema (pasajeros, conductores).

Metodo	Retorno	Descripcion
calcularDescuento()	double	Calcula el descuento base. Sobreescrito en subclases.

CLASE PASAJERO
Extiende Persona. Representa a un usuario del sistema de transporte con cedula y nombre propios.

Metodo	Retorno	Descripcion
calcularDescuento()	double	Sobreescribe el calculo de descuento.
getCedula()	String	Retorna la cedula del pasajero.
getNombre()	String	Retorna el nombre completo.
setNombre(String)	void	Actualiza el nombre del pasajero.
setCedula(String)	void	Actualiza la cedula del pasajero.
isVeterano()	boolean	Indica si el pasajero es veterano.
toString()	String	Representacion en texto del pasajero.
esValido()	boolean	Valida que los datos del pasajero sean correctos.

SUBCLASES DE PASAJERO

Clase	Atributo Adicional	calcularDescuento()	Descripcion
PasajeroRegular	Ninguno	double	Pasajero sin descuento especial. Paga tarifa completa.
PasajeroAdultoMayor	fechaNacimiento: LocalDate	double	Aplica descuento por ser adulto mayor. Verifica edad con fechaNacimiento.
PasajeroEstudiante	Ninguno	double	Aplica descuento estudiantil al calcular la tarifa.

CLASE RUTA
Representa una ruta de transporte identificada por codigo y nombre.

Atributo / Metodo	Tipo/Retorno	Visibilidad	Descripcion
codigo	String	protected	Codigo unico de laruta.
nombre	String	protected	Nombre descriptivodelaruta.
getCodigo()	String	public	Retorna el codigodelaruta.
getNombre()	String	public	Retorna el nombredelaruta.

CLASE RESERVA
Representa una reserva de viaje. Puede convertirse en Ticket al momento de abordar.

Atributo	Tipo	Descripcion
codReserva	String	Codigo unico de la reserva.
pasajero	Pasajero	Pasajero que realizo la reserva.
destino	String	Destino de la reserva.
fecha	LocalDate	Fecha programada del viaje.
vehiculo	Vehiculo	Vehiculo asignado para la reserva.

Metodo	Retorno	Descripcion
convertirTicket()	void	Convierte la reserva activa enun Ticket de viaje.
isActivo()	void	Verifica si la reserva sigue activa.
setFechaEntrada(LocalDate)	void	Actualiza la fecha de entrada/viaje.

getPasajero()	Pasajero	Retorna el pasajero asociado.
getVehiculo()	Vehiculo	Retorna el vehiculo asignado.
cancel()	void	Cancela la reserva.

CLASE TICKET

Representa el ticket de viaje generado a partir de una reserva o venta directa.

Atributo	Tipo	Descripcion
tariBasa	double	Tarifa base del ticket antes de descuentos/recargos.
fechaViaje	LocalDate	Fecha en que se realizara el viaje.
destino	String	Destino del viaje.
pasajero	Pasajero	Pasajero al que pertenece el ticket.
FESTIVO_DIA_S	LocalDate	Constante: fecha festiva para calculo de recargo.

Metodo	Retorno	Descripcion
calcTotal()	double	Calcula el total del ticket aplicando descuento del pasajero y recargo si la fecha es festiva.

CLASE FESTIVOS

Gestiona las fechas festivas del calendario. Permite verificar si una fecha dada corresponde a un festivo.

Metodo	Retorno	Descripcion
esFestivo(d: LocalDate)	boolean	Retorna true si la fecha esfestivo.
getFestivos()	List<LocalDate>	Retorna la lista completadefestivos.

La capa DAO (Data Access Object) encapsula toda la logica de lectura y escritura en archivos de texto. Cada DAO es responsable de un tipo de entidad y conoce la ruta del archivo correspondiente.

VEHICULO DAO

Atributo / Metodo	Tipo/Retorno	Descripcion
ARCHIVO_BUS	String	Ruta del archivo de buses.
ARCHIVO_MICROBUS	String	Ruta del archivo de microbuses.
ARCHIVO_BUSETA	String	Ruta del archivo de busetas.
listarVehiculos()	List<Vehiculo>	Lista todos los vehiculos de todos los archivos.
listarPorTipo(String)	List<Vehiculo>	Filtra vehiculos por tipo (Bus, MicroBus, Buseta).
obtenerVehiculo(String)	Vehiculo	Busca un vehiculo por placa.
guardarVehiculo(Vehiculo)	void	Persiste un vehiculo en el archivo correspondiente.

RESERVA DAO

Atributo / Metodo	Tipo/Retorno	Descripcion
RUTA_ARCHIVO	String	Ruta del archivo de reservas.
vehiculoDAO	VehiculoDAO	Dependencia para reconstruir vehiculos al cargar.
cargarReservas()	List<Reserva>	Carga todas las reservas desde el archivo.
cargarTicket()	List<Ticket>	Carga tickets desde el archivo.

guardarTicket(List<Reserva>)	void	Guarda la lista de reservas/tickets en archivo.
pasarATicket(String, obj)	Reserva	Convierte una reserva especifica a ticket.

TICKET DAO

Atributo / Metodo	Tipo/Retorno	Descripcion
FILE	String	Ruta del archivo de tickets.
guardar(List<Ticket>)	void	Persiste la lista de ticketsenel archivo.
listarTickets()	List<String>	Retorna representacion entexto de todos los tickets.

PASAJERO DAO

Atributo / Metodo	Tipo/Retorno	Descripcion
FILE	String	Ruta del archivo de pasajeros.
guardar(List<Pasajero>)	void	Persiste la lista de pasajeros.
buscarPorCedula(String)	Pasajero	Busca y retorna un pasajero por cedula.

FESTIVOS DAO

Atributo / Metodo	Tipo/Retorno	Descripcion
FESTIVOL_2025	List<LocalDate>	Lista precargada de festivosdel ano 2025.
esFestivo(LocalDate)	boolean	Verifica si una fecha esta enla lista de festivos.
getFestivos()	List<LocalDate>	Retorna todos los festivos registrados.

Los servicios contienen la logica de negocio del sistema. Coordinan los DAOs y aplican las reglas de negocio como validaciones, calculos y flujos de trabajo.

GESTOR RESERVAS

Es el servicio central del sistema. Orquesta la creacion, cancelacion y consulta de reservas, integrando todos los demas servicios y DAOs.

Atributo	Tipo	Descripcion
reservas	List<Reserva>	Lista en memoria de reservasactivas.
reservaDAO	ReservaDAO	Acceso a persistencia de reservas.
pasajeroDAO	PasajeroDAO	Acceso a persistencia de pasajeros.
ticketService	TicketService	Servicio para gestion de tickets.
keyCapDisp	Map<Vehiculo,List>	Mapa de capacidad disponible por vehiculo.

TICKET SERVICE

Metodo	Retorno	Descripcion
listarReservasArchivo()	List<Reserva>	Carga y retorna reservas desdeel archivo.
cargarDesdeArchivo()	int	Carga reservas a memoria, retorna cantidad cargada.
crearReserva(String,String,String,LocalDate)	Reserva	Crea una nueva reserva validando disponibilidad.
cancelarReservaTicket(String)	void	Cancela una reserva o ticket por codigo.

buscarPorConfig(String)	Reserva	Busca una reserva por codigo.
guardar()	void	Persiste todas las reservas en archivo.
Atributo / Metodo	Tipo/Retorno	Descripcion
dao	TicketDAO	DAO para persistencia de tickets.
venderTicket(String,String,String,double,LocalDate)	void	Crea y persiste un ticket de venta directa.
listarTickets()	List<String>	Lista todos los tickets en formato texto.
getTicketsFechasD()	List<Ticket>	Retorna tickets con fechas destacadas/festivas.
calcularTicketConFecha(Pasajero,String,double,LocalDate)	void	Calcula tarifa final con fecha, descuento y recargo.

PASAJERO SERVICE

Atributo / Metodo	**Tipo/Retorno**	**Descripcion**
dao	PasajeroDAO	DAO para persistenciade pasajeros.
registrarPasajero(String,String,LocalDate)	void	Registra un nuevo pasajero con nombre, cedula y fecha de nacimiento.

buscarPorCedula(String)	Pasajero	Busca y retorna un pasajero por su numero de cedula.

CLASE MENU

Atributo / Metodo	Tipo/Retorno	Descripcion
vehiculoDAO	VehiculoDAO	DAO para persistencia devehiculos.
asignarConductor(String,Persona)	void	Asigna un conductor a unvehiculo por placa.
asignarRuta(String,Ruta)	void	Asigna una ruta a un vehiculo por placa.
validarCapacidad(Vehiculo)	boolean	Verifica si hay cupo disponible en el vehiculo.
planificarVehiculo(Vehiculo)	void	Planifica la operacion del vehiculo.
calcularCantidadBoletos()	void	Calcula los boletos disponibles por vehiculo.
eliminarVehiculo(Vehiculo)	void	Elimina un vehiculo del sistema.
obtenerVehiculos()	void	Carga y muestra todos los vehiculos disponibles.

CLASE MAIN

Punto de entrada de la aplicacion. Instancia el Menu y lanza la ejecucion.

Metodo	Retorno	Descripcion
main(String[])	void	Metodo principal. Crea instanciade Menu e inicia el flujo de la aplicacion.

CLASE MENU

Gestiona la interaccion con el usuario a traves de menus en consola. Delega las operaciones a los servicios correspondientes.

Atributo / Metodo	Tipo/Retorno	Descripcion
configRuta	String	Ruta de configuracion del sistema.
ticket()	void	Submenu para gestion de tickets.
menuVehiculo()	void	Submenu para gestion de vehiculos.
menuReserva()	void	Submenu para gestion de reservas.
menuPasajero()	void	Submenu para gestion de pasajeros.
menuEstadisticas()	void	Submenu para consultar estadisticas del sistema.

RELACIONES ENTRE CLASES

A continuacion se resumen las relaciones mas importantes identificadas en el diagrama UML:

Relacion	Origen	Destino	Tipo	Descripcion
Herencia	Bus	Vehiculo	Generalizacion	Bus extiende Vehiculo.
Herencia	MicroBus	Vehiculo	Generalizacion	MicroBus extiende Vehiculo.
Herencia	Buseta	Vehiculo	Generalizacion	Buseta extiende Vehiculo.
Herencia	Pasajero	Persona	Generalizacion	Pasajero extiende Persona.
Herencia	PasajeroRegular	Pasajero	Generalizacion	PasajeroRegular extiende Pasajero.

Herencia	PasajeroAdultoMayor	Pasajero	Generalizacion	PasajeroAdultoMayor extiende Pasajero.
Herencia	PasajeroEstudiante	Pasajero	Generalizacion	PasajeroEstudiante extiende Pasajero.
Implementa	Vehiculo	Imprimible	Realizacion	Vehiculo implementa la interfaz Imprimible.
Implementa	Vehiculo	Calculable	Realizacion	Vehiculo implementa la interfaz Calculable.
Asociacion	Vehiculo	Ruta	Composicion	Un Vehiculo tiene una Ruta asignada.
Asociacion	Reserva	Pasajero	Asociacion	Una Reserva referencia a un Pasajero.
Asociacion	Reserva	Vehiculo	Asociacion	Una Reserva referencia a un Vehiculo.
Asociacion	Ticket	Pasajero	Asociacion	Un Ticket referencia a un Pasajero.
Dependencia	GestorReservas	ReservaDAO	Uso	GestorReservas usa ReservaDAO.
Dependencia	GestorReservas	PasajeroDAO	Uso	GestorReservas usa PasajeroDAO.
Dependencia	GestorReservas	TicketService	Uso	GestorReservas delega en TicketService.
Dependencia	ReservaDAO	VehiculoDAO	Uso	ReservaDAO usa VehiculoDAO para reconstruir vehiculos.

Dependencia	Menu	GestorReservas	Uso	Menu orquesta a traves de GestorReservas.
Dependencia	Menu	ServicioVehiculo	Uso	Menu delega gestion de vehiculos.
Dependencia	Menu	PasajeroService	Uso	Menu delega gestion de pasajeros.
Dependencia	Menu	TicketService	Uso	Menu delega gestion de tickets.
Dependencia	Main	Menu	Uso	Main instancia y lanza el Menu.